Lecture 26: SQL Aggregation (Exercises)

COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING

Z2004: Database Management Systems

IIT Madras Zanzibar

Thursday, 14 May 2026

Today: Aggregation Practice (Write by Hand)

Goal: prove we already covered aggregation in L25 by practicing it.

- You will write SQL on paper.
- Work in pairs. Discuss logic first, then write the query.
- Then we check solutions together.

Reminder (from L25):

- Aggregates: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`
- Grouping: `GROUP BY`
- Filtering groups: `HAVING`

Dataset for Exercises (SchoolDB)

Assume these tables exist:

Student(SID, Name, Dept, GPA)

SID	Name	Dept	GPA
1	Asha Ali	CSE	3.80
2	Zain Omar	EEE	3.45
3	Maryam Hassan	ME	3.12
4	Hamid Musa	CSE	2.70
5	Fatima Noor	CSE	*NULL*
6	Omar Ali	ME	2.90

Course(CID, CName, Credits, Dept)

CID	CName	Credits	Dept
10	DBMS	3	CSE
11	Signals	4	EEE
12	Thermo	3	ME
13	AI	3	CSE

Note: `NULL` GPA means unknown; aggregates like `AVG(GPA)` ignore NULLs.

Warm-up (Individual, 3 min)

Write SQL queries for each:

1. How many students are there in total?
2. How many students have a non-NULL GPA?
3. What is the minimum and maximum GPA (ignoring NULLs)?
4. What is the average GPA (ignoring NULLs)?

Rule: only `SELECT` + aggregates (no `GROUP BY` yet).

Group By Practice (Pairs, 6 min)

Write SQL queries for each:

1. Dept-wise student count: show `Dept, NumStudents`.
2. Dept-wise average GPA (ignore NULL GPAs).
3. Dept-wise minimum GPA and maximum GPA.

Hint:

- Use `GROUP BY Dept`.
- Use aliases like `AS NumStudents`.

HAVING Practice (Pairs, 6 min)

Write SQL queries for each:

1. Show only departments with at least 2 students.
2. Show departments whose average GPA is at least 3.0.
3. Show departments with at least 2 known GPAs (non-NULL).

Reminder:

- `WHERE` filters rows before grouping.
- `HAVING` filters groups after grouping.

Can WHERE and HAVING be used together?

Yes. They apply at different stages:

- `WHERE`: filters **rows** before `GROUP BY`.
- `HAVING`: filters **groups** after aggregation.

Example: ignore NULL GPAs, then keep only departments with average GPA ≥ 3.0.

```
SELECT Dept, AVG(GPA) AS AvgGPA
FROM Student
WHERE GPA IS NOT NULL
GROUP BY Dept
HAVING AVG(GPA) >= 3.0;
```

More Aggregation Practice (Pairs, 8 min)

Use only `Student` and `Course`.

1. Total credits offered across all courses.
2. Dept-wise total credits offered: show `Dept, TotalCredits`.
3. Dept-wise average credits per course.
4. Show only departments offering at least 2 courses.
5. Show only departments whose total credits is at least 6.

Hint:

- Use `SUM(Credits)`, `AVG(Credits)`, `COUNT(*)`.
- Use `GROUP BY Dept` and `HAVING ...`.

Solutions: Warm-up

```sql
-- 1) Total students
SELECT COUNT(*) AS TotalStudents
FROM Student;

-- 2) Students with known GPA
SELECT COUNT(GPA) AS StudentsWithGPA
FROM Student;

-- 3) Min/Max GPA (NULL ignored automatically)
SELECT MIN(GPA) AS MinGPA, MAX(GPA) AS MaxGPA
FROM Student;

-- 4) Average GPA (NULL ignored)
SELECT AVG(GPA) AS AvgGPA
FROM Student;
```

Solutions: GROUP BY

```sql
-- 1) Dept-wise student count
SELECT Dept, COUNT(*) AS NumStudents
FROM Student
GROUP BY Dept;

-- 2) Dept-wise average GPA (NULL ignored)
SELECT Dept, AVG(GPA) AS AvgGPA
FROM Student
GROUP BY Dept;

-- 3) Dept-wise min/max GPA
SELECT Dept, MIN(GPA) AS MinGPA, MAX(GPA) AS MaxGPA
FROM Student
GROUP BY Dept;
```

Solutions: HAVING

```sql
-- 1) Departments with at least 2 students
SELECT Dept, COUNT(*) AS NumStudents
FROM Student
GROUP BY Dept
HAVING COUNT(*) >= 2;

-- 2) Departments with average GPA >= 3.0
SELECT Dept, AVG(GPA) AS AvgGPA
FROM Student
GROUP BY Dept
HAVING AVG(GPA) >= 3.0;

-- 3) Departments with at least 2 known GPAs
SELECT Dept, COUNT(GPA) AS KnownGPAs
FROM Student
GROUP BY Dept
HAVING COUNT(GPA) >= 2;
```

Solutions: More Aggregation Practice (1/2)

```sql
-- 1) Total credits offered across all courses
SELECT SUM(Credits) AS TotalCredits
FROM Course;

-- 2) Dept-wise total credits offered
SELECT Dept, SUM(Credits) AS TotalCredits
FROM Course
GROUP BY Dept;

-- 3) Dept-wise average credits per course
SELECT Dept, AVG(Credits) AS AvgCredits
FROM Course
GROUP BY Dept;
```

Solutions: More Aggregation Practice (2/2)

```sql
-- 4) Only departments offering at least 2 courses
SELECT Dept, COUNT(*) AS NumCourses
FROM Course
GROUP BY Dept
HAVING COUNT(*) >= 2;

-- 5) Only departments whose total credits is at least 6
SELECT Dept, SUM(Credits) AS TotalCredits
FROM Course
GROUP BY Dept
HAVING SUM(Credits) >= 6;
```

Summary: SQL Aggregation (L25 & L26)

Aggregate functions take many rows and return a **single value per group**.

- `COUNT(*)`: counts rows (includes rows where columns are NULL).
- `COUNT(col)`: counts **non-NULL** values in `col`.
- `SUM(col)`: adds numeric values in `col` (NULLs ignored).
- `AVG(col)`: average of numeric values in `col` (NULLs ignored).
- `MIN(col)` / `MAX(col)`: smallest / largest value in `col` (NULLs ignored).

How they work

- Without `GROUP BY`: one result row for the whole table.
- With `GROUP BY Dept`: one result row **per Dept**.
- `WHERE` filters rows **before** grouping; `HAVING` filters groups **after** grouping.